



UNIX SHELL

Group 09

CORE ARCHITECTURE

User Programs (shell.c)



System Call Layer (syscall.c)



Process Management (proc.c)



File System / Disk Support (fs)



Device Drivers (console, IDE)

CORE ARCHITECTURE

User Processes – user/shell.c

- Reads input via `read(STDIN, buf, size)`
- Parses commands and arguments
- Executes commands using `fork()` + `exec()`
- handles built-in functions like `cd` or `exit`

```
pid = fork();  
if (pid == 0) exec(cmd, argv);  
else wait(NULL);
```

System Call Interface – kernel/syscall.c

- Receives trap from user space
- Identifies syscall number
- Calls corresponding kernel function

```
case SYS_pipe:  
return sys_pipe(fd_array);
```

CORE ARCHITECTURE

Process Manager – kernel/proc.c

- **do_fork()** → creates new process with its PCB
- **do_exec()** → loads program binary into process memory
- **do_wait()** → blocks parent until child process exits

```
pid = do_fork();    // kernel allocates PCB  
do_exec(pid, program); // loads binary into process  
do_wait(pid);      // parent waits for child
```

CORE ARCHITECTURE

File System / Disk Support – fs/

- open(), read(), write() syscalls
- Directory operations: mkdir(), opendir(), readdir()
- Supports pipe buffers for inter-process communication

```
int fd = open("file.txt", O_RDONLY);  
read(fd, buffer, size);
```

Device Drivers (Console, IDE) – dev/

- Console: displays shell prompt, reads keyboard input
- Disk: reads/writes files as requested by file system

```
write(STDOUT, "$ ", 2); // shell prompt  
read(STDIN, buf, 128); // user input
```




KEY COMPONENTS

● Shell User Program

- Implemented in user/shell.c
- Contains main shell logic

● Built-in Commands

- Used to modify shell's own state
- Example (cd,exit)

● External Command Handler

- Executes user programs
- Implemented using syscalls:
(fork(), exec(), wait())
 - Used for commands like
(ls, cat , mkdir)

● Pipe Handler

- Supports multiple process creation
- Uses system calls:
pipe()
dup2()

KEY COMPONENTS

● Syscall Interface

- Implemented in syscall.c
- Acts as bridge between shell and kernel

● Process Manager

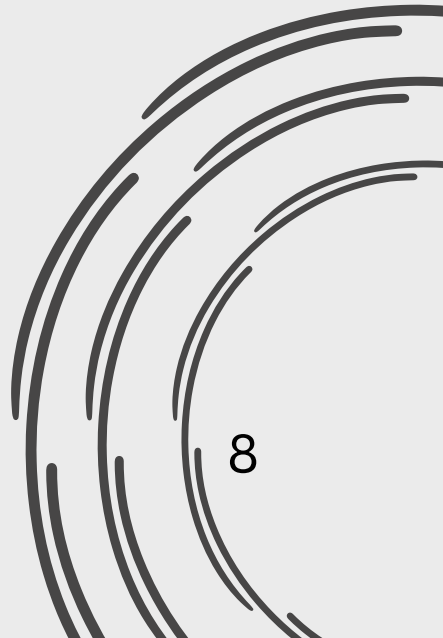
- Tracks file descriptors for pipe communication
- supports (fork , exec , wait)

● File System Support

- Enables file I/O operations
- Used by commands such as: (cat , ls , mkdir)



IMPLEMENTATION IDEA





PROCESS-BASED EXECUTION

Purpose

Isolate processes, allow independent execution, and support piped commands.

fork()

Creates child processes

wait()

Parent process waits for child to maintain sequential implementation



PROCESS-BASED EXECUTION

```
function execute_command(command, arguments):  
  
    pid = fork()                // create child process  
  
    if pid < 0 then  
        print "Fork failed"  
        return  
  
    else if pid == 0 then        // CHILD_toggle  
        // child executes program  
        exec(command, arguments)  
  
        // if exec fails  
        print "Execution failed"  
        exit()  
  
    else                        // PARENT_process  
        // parent waits for child to finish  
        wait(pid)  
  
        print "Command finished"
```



PIPE-BASED INTER-PROCESS COMMUNICATION (IPC)

Purpose

Enable commands like `cat file | grep pattern` to work

pipe()

syscall to create a kernel-managed buffer with two file descriptors

dup2()

Child processes use to redirect their standard input/output to the pipe.

PIPE-BASED INTER-PROCESS COMMUNICATION (IPC)

```
function execute_pipe(cmd1, args1, cmd2, args2):

    create pipe fd[2]          // fd[0] = read end, fd[1] = write end

    pid1 = fork()

    if pid1 == 0 then
        // CHILD 1 : Producer (cmd1)

        close(fd[0])           // not reading
        dup2(fd[1], STDOUT)    // redirect output to pipe
        close(fd[1])

        exec(cmd1, args1)      // run first command
        exit()

    pid2 = fork()

    if pid2 == 0 then
        // CHILD 2 : Consumer (cmd2)

        close(fd[1])           // not writing
        dup2(fd[0], STDIN)     // read from pipe
        close(fd[0])

        exec(cmd2, args2)      // run second command
        exit()

    // PARENT PROCESS
    close(fd[0])
    close(fd[1])

    wait(pid1)
    wait(pid2)
```



SHELL COMMAND PARSING

Purpose

Transform raw input string into structured commands

Tokenization

Tokenize commands and arguments from user

Supports

- Built-in commands (cd, exit)
- External commands (fork+exec)
- Pipelines (|)
- Redirection (>, <)



SHELL COMMAND PARSING

```
function parse_command(input_line):

    tokens = split(input_line, " ")    // tokenize by spaces

    command_list = empty list
    current_command = new command

    for each token in tokens:

        if token == "|" then
            add current_command to command_list
            current_command = new command

        else if token == ">" then
            current_command.output_redirect = next token

        else if token == "<" then
            current_command.input_redirect = next token

        else
            if current_command.name is empty then
                current_command.name = token
            else
                add token to current_command.arguments

    add current_command to command_list

    return command_list
```



SYSCALL-DRIVEN KERNEL INTERACTION

Purpose

Interacts with kernel on request from shell

Syscalls

- `fork()`
- `exec()`
- `wait()`
- `pipe()`
- `read()`
- `write()`

Does

create processes, execute programs, manages I/O, communicate between processes

SYSCALL-DRIVEN KERNEL INTERACTION

How shell requests kernel to perform operations

```
function run_shell():  
    while true:  
        write(STDOUT, "$ ")           // show prompt  
        input = read(STDIN)           // get user command  
        commands = parse_command(input)  
        if commands contain pipe then  
            execute_pipe(commands)  
        else  
            execute_single(commands)
```

Inside execute_single()

```
function execute_single(cmd):  
    pid = syscall_fork()  
    if pid == 0 then                    // CHILD  
        syscall_exec(cmd.name, cmd.args)  
        syscall_exit()  
    else                                // PARENT  
        syscall_wait(pid)
```

What Happens in Kernel

```
on_syscall(syscall_number, arguments):  
    switch syscall_number:  
        case FORK:  
            create_process()  
        case EXEC:  
            load_program_into_memory()  
        case WAIT:  
            block_parent_until_child_finishes()  
        case READ:  
            read_from_keyboard_or_file()  
        case WRITE:  
            output_to_screen_or_file()
```


Modules We Extend or Modify

syscall.c / syscall.h

- Added syscall numbers and dispatch entries for pipe(), dup2(), and required shell syscalls.

init.c

- Modified system startup to launch the shell as the first user process.

Modules We Extend or Modify

proc.c

- Extended process management to handle fork/exec/wait and track multiple pipe file descriptors.

file.c / Pipe Module

- Implemented kernel pipe buffers to enable inter-process communication.



INTEGRATION WITH MCERTIKOS

User-Level Interaction

- Reads input, parses commands
- Determines built-in vs external commands
- Uses syscalls: `fork()`, `exec()`, `wait()`, `pipe()`, `dup2()`
- Syscalls trigger kernel traps for safe execution



INTEGRATION WITH MCERTIKOS

Kernel Interaction

- **Process Manager (proc.c):** fork, exec, wait
- **File System (fs/):** open, read, write for commands & pipes
- **Pipe Buffers / FDs:** kernel handles buffering & blocking
- **Scheduler (sched.c):** manages execution, blocks/unblocks processes



THANK YOU